

Hackathon Project Phases Template that ensures students can complete it efficiently while covering all six phases. The template is structured to capture essential information without being time-consuming.

Hackathon Project Phases Template

Project Title: JobSwift

Team Name: Inquisitive

Team Members:

- Ch.Leela Madhuri
 - B.Lavanya
 - Ch,Mahathi
 - A.Anitha
 - D.Lavanya
-

Phase-1: Brainstorming & Ideation

Objective:

The objective of a job swift (or job shift) typically refers to the goals or reasons behind changing jobs or transitioning to a new role. These objectives can vary depending on the individual's personal and professional aspirations. Here are some common objectives of a job shift:

- **Career Growth:** Seeking better opportunities for advancement or promotions. Moving into a role with more responsibilities or leadership potential.
- **Skill Development:** Gaining new skills or expertise in a different field or industry. Transitioning to a role that aligns better with one's long-term career goals.
- **Higher Compensation:** Pursuing a job with better pay, benefits, or financial incentives.
- **Work-Life Balance:** Moving to a role with more flexible hours, remote work options, or a better work .
- **Job Satisfaction:** Switching to a role that aligns more closely with personal interests, passions, or values. Escaping a toxic or unfulfilling work environment.
- **Industry or Role Change:** Transitioning to a different industry or job function to explore new opportunities.

Key Points:

1. Problem statement:

- Career Dissatisfaction and Stagnation
- Difficulty in Career Transitions
- Lack of Personalized Guidance
- Work-Life Imbalance
- Inefficient Job Search Process
- Fear of Risk and Uncertainty

2. Proposed solution:

- **Smart Job Matching:** Use AI to recommend jobs that closely match user profiles, reducing time spent searching.
- **Transparent Job Listings:** Ensure job descriptions include clear details about roles, expectations, and company culture.
- **Application Tracking:** Provide tools to track job applications, interviews, and follow-ups in one place.
- **Job Filters for Work-Life Balance:** Enable users to filter job opportunities based on criteria like remote work, flexible hours, and company culture.
- **Wellness Resources:** Provide access to resources on mental health, stress management, and work-life integration.
- **Employer Partnerships:** Collaborate with companies that prioritize employee well-being and highlight them on the platform.

3. Target users:

1. Job Seekers:

- **Recent Graduates:** Individuals who have recently completed their education and are seeking entry-level positions.
- **Experienced Professionals:** Those looking to advance their careers, switch industries, or find new opportunities.
- **Freelancers/Contract Workers:** Individuals seeking short-term or project-based work.
- **Career Changers:** People looking to transition into a new field or industry.
- **Students:** Those seeking internships or part-time jobs while studying.

4. Expected outcomes:

The expected outcome for JobSwift is to streamline the job search and recruitment process, enabling job seekers to find suitable roles quickly and employers to hire top talent efficiently, ultimately enhancing user satisfaction and engagement.

Phase-2: Requirement Analysis

Objective:

1. To **identify and prioritize the needs, expectations, and pain points of both job seekers and employers** to design a user-friendly, efficient, and scalable platform that simplifies job matching, enhances recruitment processes, and delivers a seamless experience for all stakeholders.

•

Key Points:

1. Technical Requirements:

- Programming Language: **python**
- Backend: **Python (Django/Flask), Node.js**
- Frontend: **JavaScript (React.js, Angular, or Vue.js)**
- Database: **SQL (PostgreSQL/MySQL) or NoSQL (MongoDB)**

2. Functional Requirements:

- **User Profiles:** Job seekers create resumes; employers post jobs.
- **Job Search & Filters:** Advanced search by role, location, salary.
- **AI Matching:** Recommend jobs to seekers, candidates to employers.
- **Application & Messaging:** Apply, track status, and communicate directly.

3. Constraints & Challenges:

- **Scalability:** Handling high traffic and large datasets as the user base grows.
 - **Data Security:** Ensuring user data privacy and compliance with GDPR/CCPA.
 - **AI Accuracy:** Developing reliable AI/ML algorithms for job matching.
 - **User Experience:** Maintaining a seamless, fast, and intuitive interface across devices.
 - **Cost Management:** Balancing development, infrastructure, and maintenance costs.
 - **Competition:** Differentiating from established job platforms like LinkedIn or Indeed.
 - **Adoption:** Attracting both job seekers and employers to the platform initially
-

Phase-3: Project Design

Objective:

- [User Interface] --> [Application Layer] --> [Business Logic Layer] --> [Data Layer]
- ↑ ↓
- [Third-Party APIs] <-- [Integration Layer]
- ↑
- [Infrastructure Layer]

Key Points:

❖ **System Architecture Diagram:**

1. User Interface (UI)

- **Job Seekers:** Search jobs, apply, and track applications.
- **Employers:** Post jobs and manage applications.
- **Admin:** Manage platform settings and users.

2. Application Layer

- Handles authentication, job search, application tracking, and notifications.

3. Business Logic Layer

- **AI/ML:** Job matching and resume parsing.
- **Analytics:** Tracks user engagement and platform performance.

4. Data Layer

- **Database:** Stores user profiles, jobs, and applications.
- **Cache:** Speeds up frequent data access.
- **File Storage:** Stores resumes and documents.

5. Integration Layer

- Connects with payment gateways, third-party APIs (e.g., LinkedIn), and AI services.

❖ **User Flow:**

- **Job Seekers:** Sign up, search for jobs using filters, apply with resumes, and track application status.
- **Employers:** Register, post job listings, review applications, and communicate with candidates.
- **Admins:** Manage users, monitor platform activity, and resolve issues.
- **Notifications:** Users receive alerts for job matches, application updates, and messages.

❖ **UI/UX Considerations:** (If applicable, wireframe or basic layout)

1. Simplicity & Clarity

- Clear calls-to-action (e.g., "Apply Now," "Save Job").
- Minimalistic forms for job applications and profile creation.

2. Responsive Design

- Optimized for mobile, tablet, and desktop

3. Personalization

- Tailored job recommendations based on user preferences and behavior.
- Customizable dashboards for job seekers and employers.

4. Speed & Performance

- Fast loading times for job searches and application submissions.
- Progress indicators for lengthy processes (e.g., resume uploads).

5. Visual Hierarchy

- Highlight important elements (e.g., job titles, deadlines).
- Use icons, colors, and typography to guide users.

6. Feedback & Notifications

- Real-time feedback for actions (e.g., "Application Submitted").
- Notifications for job alerts, messages, and updates.

8. Error Handling

- Clear error messages with actionable solutions.
- Validate forms in real-time to prevent mistakes.

9. Onboarding & Help

- Interactive onboarding for new users.
- Tooltips, FAQs, and support options for guidance.

Phase-4: Project Planning (Agile Methodologies)

Objective:

Sprint	Task	Priority	Duration	Deadline	Assigned to	Dependence	Expected Outcome
Sprint -1	Design Login/Registration UI	High	1 day	Day 1	Frontend Developer	None	Functional login/registration UI
Sprint -1	Develop Authentication API	High	1 day	Day 1	Backend Developer	None	Secure authentication API
Sprint -1	Integrate Auth UI with API	High	1 day	Day 1.5	Full-Stack Developer	Login UI, Auth API	Seamless login/registration flow
Sprint -2	Test Authentication Flow	High	1 day	Day 1.5	QA Engineer	Integrated Auth Flow	Bug-free authentication
Sprint -2	Design Job Search UI	High	1 day	Day 1	Frontend Developer	None	User-friendly job search interface
Sprint -2	Integrate Job Search UI with API	High	1 day	Day 1	Backend Developer	None	Efficient job search API
Sprint -3	Test Job Search Feature	High	1 day	Day 1	Fullstack developer	Job Search UI, Job Search API	Functional job search feature
Sprint -3	Design Resume Upload UI	Medium	1 day	Day 1	Frontend Developer	None	Secure resume storage API
Sprint -3	Test Resume Upload Feature	Medium	1 day	Day 1.5	QA Engineer	None	Bug-free resume upload
Sprint -4	Design Employer Dashboard UI	Medium	1 day	Day 1.5	Fullstack developer	None	Clean employer dashboard UI
Sprint -4	Develop Job Posting API	Medium	1 day	Day 1	Backend Developer	None	Efficient job posting API
Sprint -4	Develop Notification API	Low	1 day	Day 1	Backend Developer	Notification UI, API	Efficient notification API
Sprint -4	Integrate Notification Feature	Low	1 day	Day 1.5	Fullstack developer	Notification UI, API	Functional notification system

- ✓ Frontdeveloper- Ch.Leela Madhuri
- ✓ Backenddeveloper-Ch.Mahathi,B.Lavanya
- ✓ QA Engineer-D.Lavanya

Phase-5: Project Development

Objective:

- **JobSwift** is a **modern job search and recruitment platform** that leverages **AI/ML** to match job seekers with the right opportunities while providing employers with tools to streamline hiring.

❖ Key Points:

1. Technology Stack Used:

- **Frontend:** Streamlit web Framework
- **Backend:** python framework like Fast API, flask or Django (Python)
- **Database:** Streamlit UI
- **Development Process:**
 - **Requirement Analysis:** Gather and define stakeholder needs and features.
 - **Design & Prototyping:** Create wireframes and prototypes for feedback.
 - **Tech Stack Setup:** Configure frontend, backend, databases, and cloud hosting.
 - **Agile Development:** Build features incrementally in sprints.
 - **Coding & Integration:** Write code and integrate frontend, backend, and third-party services.
 - **Testing:** Perform unit, integration, and user acceptance testing.
 - **Deployment:** Deploy to production using CI/CD pipelines.
 - **Maintenance & Updates:** Regularly update, fix issues, and scale infrastructure.

2. Challenges & Fixes:

- **Challenge: Slow job search performance** due to large database.
- **Fix:** Implemented **caching (Redis)** and optimized database queries.
- **Challenge: Resume parsing errors** with inconsistent formats.
Fix: Used **AI/ML libraries (NLTK, TensorFlow)** to improve parsing accuracy.
- **Challenge: User authentication security risks.**
Fix: Integrated **OAuth 2.0** and **JWT** for secure authentication.
- **Challenge: Real-time notification delays.**
Fix: Used **WebSocket** and **push notification services (Twilio)** for instant updates.

Phase-6: Functional & Performance Testing

Objective:

❖ **Functional testing:**

- Verified all features (e.g., job search, resume upload, notifications) work as intended.
- Tested user flows (e.g., registration, application submission) for accuracy.

❖ **Performance Testing:**

- Checked system response under load using tools like JMeter.
- Optimized database queries and caching for faster performance.
- Ensured scalability with cloud infrastructure (AWS/GCP).
-

Key Points:

Test case id	Category	Test scenario	Expected Outcome	Status	Tester
TC-001	Authentication	User registration.	Account created	Pass	QA 1
TC-002	Authentication	User login (correct credentials).	Login successful.	Pass	QA 1
TC-003	Authentication	User login (wrong credentials)	Error message.	fail	QA 2
TC-004	Job Search	Search with filters.	Relevant jobs shown.	Needs Optimisation	QA 2
TC-005	Application	Upload resume and apply.	Application submitted.	Pass	QA 3
TC-006	Application	Apply without resume.	Error message.	Fail-doesnot not show error message	QA 4
TC-007	Employer	Post a job.	Job listed on platform.	Pass	QA 4
TC-008	Employer	View applications.	Applications displayed.	Pass	QA 1
TC-009	Notifications	Job seeker gets job alert	Notification received.	Pass	QA 2
TC-010	Notifications	Employer gets new application alert.	Notification received.	Pass	QA 3
TC-011	Performance	Job search	Fast results.	Pass	QA 2

		under 2 seconds			
TC-012	Performance	1,000 users using system.	No crashes or delays.	Needs optimization	QA 1

Final Submission

- 1. **Project Report Based on the templates**
 - 2. **Demo Video (3-5 Minutes)**
 - 3. **GitHub/Code Repository Link**
 - 4. **Presentation**
-